MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1

Software Design

Author: Iulia Maria Martin
Group: 30433

Faculty of Automation and Computer Science

March, 2026

Contents

0.1 Problem definition . . . 2

1 Analysis phase - Use cases & requirements . . . 2

1.1 Functional requirements . . . 2

1.2 Use cases . . . 2

2 Design Phase . . . 4

2.1 Architecture Overview . . . 4

2.2 Entity-Relationship Diagram . . . 4

2.3 Component X - Web Application Server . . . 5

2.4 Component Y - Database Layer . . . 5

2.5 Design Patterns . . . 6

2.6 Class Diagrams . . . 6

2.7 Sequence Diagrams . . . 7

3 Implementation Phase - Application . . . 8

3.1 Tools and Technologies . . . 8

3.2 Application Structure . . . 8

3.3 User Manual . . . 9

4 Testing and Validation . . . 10

0.1 Problem definition

This project implements a comprehensive Flower Shop Management System that manages the complete workflow of a flower retail business. The application provides customers with the ability to browse and purchase flowers and bouquets while offering administrative tools for inventory management, order processing, and user role-based access control. The system is built using ASP.NET Core with Entity Framework Core for persistence and MySQL as the database backend, following a layered MVC architecture with clear separation of concerns between Controllers, Services, and Repositories.

Development Phases

The project was implemented in five phases: Phase 1 defined the domain model (Flower, Bouquet, User, etc.), while Phase 2 established the Repository Pattern for data persistence. Phase 3 focused on business logic, validation, and authentication, followed by Phase 4, which developed the MVC web application. Finally, Phase 5 integrated advanced features like session-based cart management and complex order workflows.

Additional Features and Deliverables

The system includes advanced search and filtering, multi-column sorting, and session-based shopping cart management. Deliverables consist of the layered source code, database migrations, and technical documentation. Bonus features include secure SHA256 password hashing, HTTP-only session cookies, and comprehensive error-handling mechanisms to provide a professional user experience.

1 Analysis phase - Use cases & requirements

1.1 Functional requirements

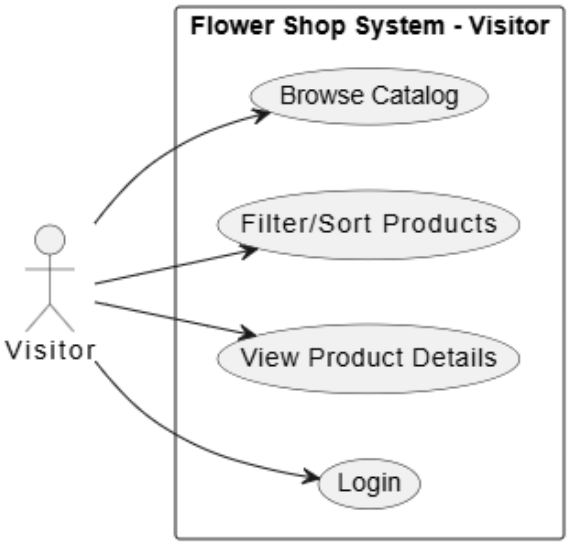
The system fully implements all required functional specifications. CRUD operations are available for all entities with proper validation and error handling. Role-based access control is enforced through authentication middleware and authorization attributes, ensuring users can only access permitted functionality. The MVC architecture maintains strict separation with controllers handling only request coordination, services implementing complete business logic, repositories managing data access, and entities representing the rich domain model. Repository implementation includes custom Entity Framework queries with optimized performance, dynamic filtering capabilities, and proper relationship management using Include and ThenInclude patterns.

1.2 Use cases

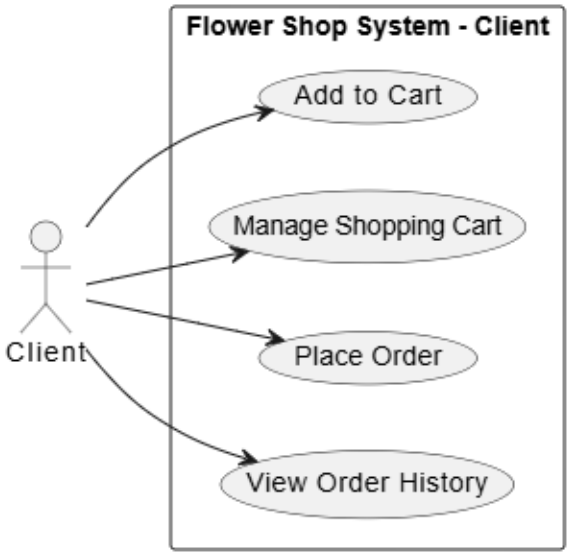
Use cases describe in text the interactions between actors and the system in order to accomplish specific tasks.

- **Browse Catalog** - Visitor/Client views flowers and bouquets with filtering and sorting

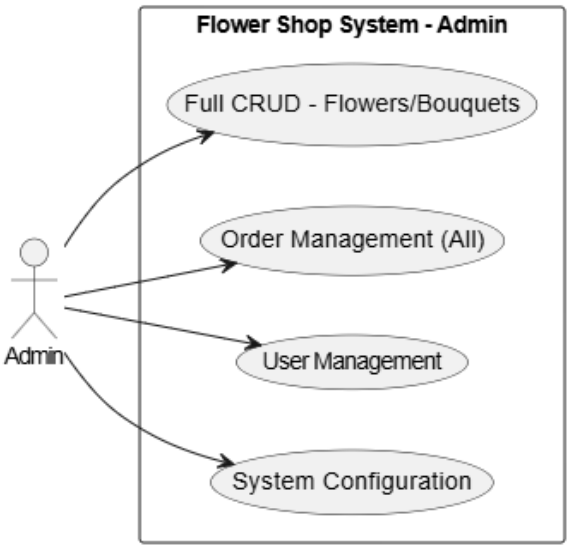
- **View Product Details** - Visitor/Client views detailed information about specific products
- **Add to Cart** - Client adds items to shopping cart with quantity selection
- **Place Order** - Client converts cart items into a formal order with delivery info
- **Manage Products** - Admin creates, edits, and deletes flowers and bouquets
- **Process Orders** - Florist updates order status and manages assignments
- **User Authentication** - Visitor logs in to access personalized features
- **Order Management** - Admin views and manages all system orders



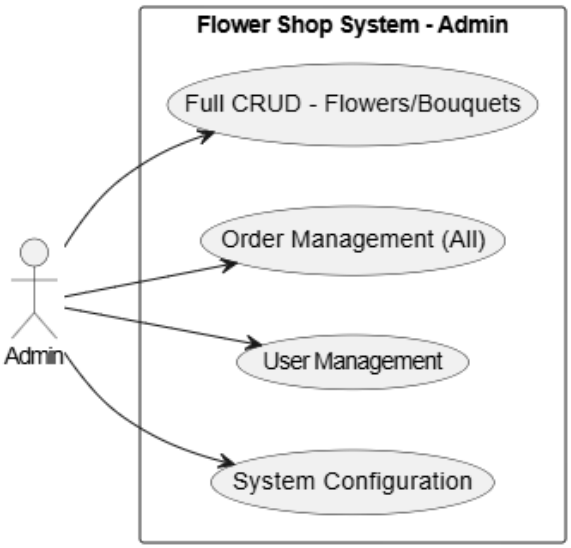
(a) Visitor Diagram



(b) Client Diagram



(a) Florist Diagram



(b) Admin Diagram

2 Design Phase

2.1 Architecture Overview

The application follows a **layered client-server architecture** with clear separation of concerns. The client-side consists of web browsers accessing the application through HTTP/HTTPS protocols. The server-side implements ASP.NET Core MVC pattern with three distinct layers: Presentation (Controllers and Views), Business Logic (Services), and Data Access (Repositories with Entity Framework Core).

This architectural style provides several advantages: maintainability through separation of concerns, testability via dependency injection, scalability through modular design, and security through layered authorization. The main disadvantage is increased complexity, but this is justified by improved code organization and long-term maintainability.

2.2 Entity-Relationship Diagram

Users table: Stores user accounts with authentication data and role information. Important attributes include email (unique), password_hash (SHA256), and role (Client/Florist/Admin). Related to Orders and Florists tables.

Florists table: Contains florist profiles with contact information. Links to Users table for account association and to Flowers table for product management.

Categories table: Organizes flowers into logical groups. Simple structure with name and description, has one-to-many relationship with Flowers.

Flowers table: Core product entity with pricing, stock, and metadata. Links to Categories and Florists, participates in many-to-many relationships with Bouquets.

Bouquets table: Predefined flower arrangements with fixed pricing. Links to Orders and has many-to-many relationships with Flowers through the junction table.

Bouquet_Flowers table: Junction table implementing many-to-many relationships between Bouquets and Flowers with quantity information.

Orders table: Customer orders with status tracking and delivery information. Links to Users, optional Bouquets, and assigned Florists.

Order_Items table: Individual items within orders, supporting either flowers or bouquets. Maintains unit prices for historical accuracy.

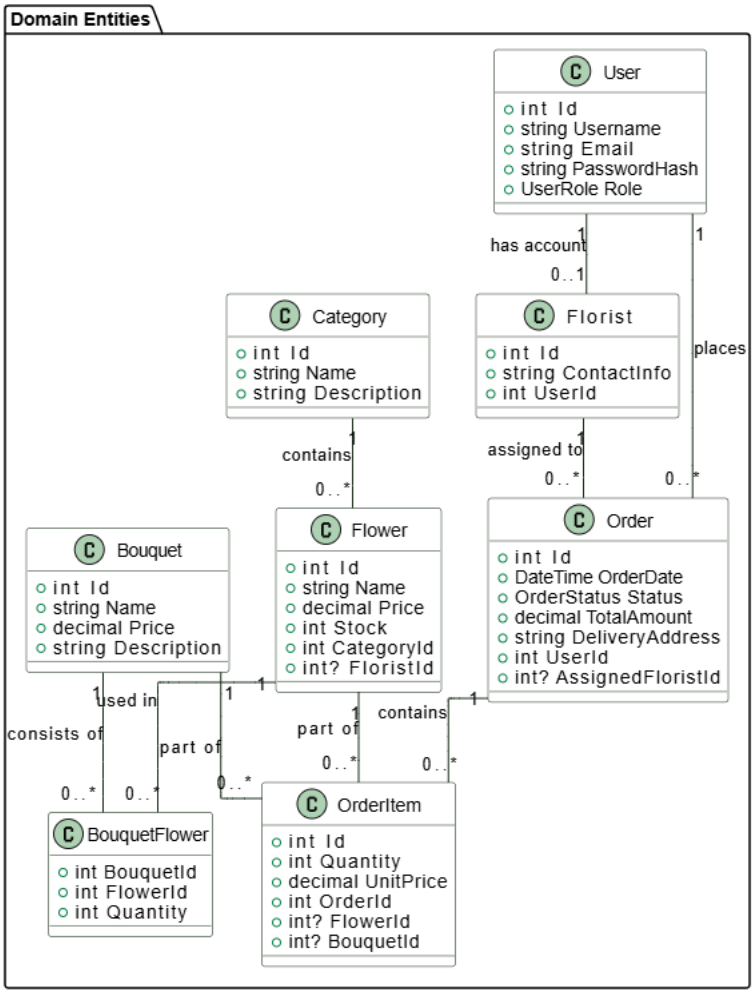


Figure 3: Entity-Relationship Diagram

2.3 Component X - Web Application Server

The web application server is the main backend component built with ASP.NET Core. It handles HTTP requests, manages user sessions, coordinates business logic, and renders HTML responses. The server uses Entity Framework Core for database operations and provides web endpoints for CRUD functionality. Security is implemented through authentication middleware and role-based authorization.

2.4 Component Y - Database Layer

The database layer consists of a MySQL server with Entity Framework Core ORM. This component handles data persistence, query optimization, and transaction management. It provides abstraction over SQL operations through repository pattern implementation and maintains referential integrity through foreign key constraints. The database supports concurrent connections and uses proper indexing for performance.

2.5 Design Patterns

The system implements several key design patterns to ensure code quality and maintainability.

- **Repository Pattern** (Structural): Solves data access abstraction by providing interface-based data operations. Involves Repository interfaces (IFlowerRepository) and concrete implementations (FlowerRepository) that use Entity Framework Core.
- **Dependency Injection** (Creational): Solves loose coupling and testability by injecting dependencies through constructors. Implemented through ASP.NET Core's built-in DI container with service registration in Program.cs.
- **Service Layer Pattern** (Architectural): Separates business logic from data access. Involves Service interfaces (IFlowerService) and implementations (FlowerService) that coordinate between controllers and repositories.

2.6 Class Diagrams

Class diagrams describe the static structure of the system by presenting classes, attributes, methods, and relationships between classes.

The Flower class represents the core product entity with properties for name, price, stock, and relationships to Category and Florist. The FlowerRepository implements IFlowerRepository interface, providing data access methods like GetByIdAsync and GetAllWithFiltersAsync. The FlowerService contains business logic, coordinating between FlowerRepository and FlowersController. The FlowersController handles HTTP requests, using dependency injection to receive IFlowerService and returning appropriate IActionResult responses.

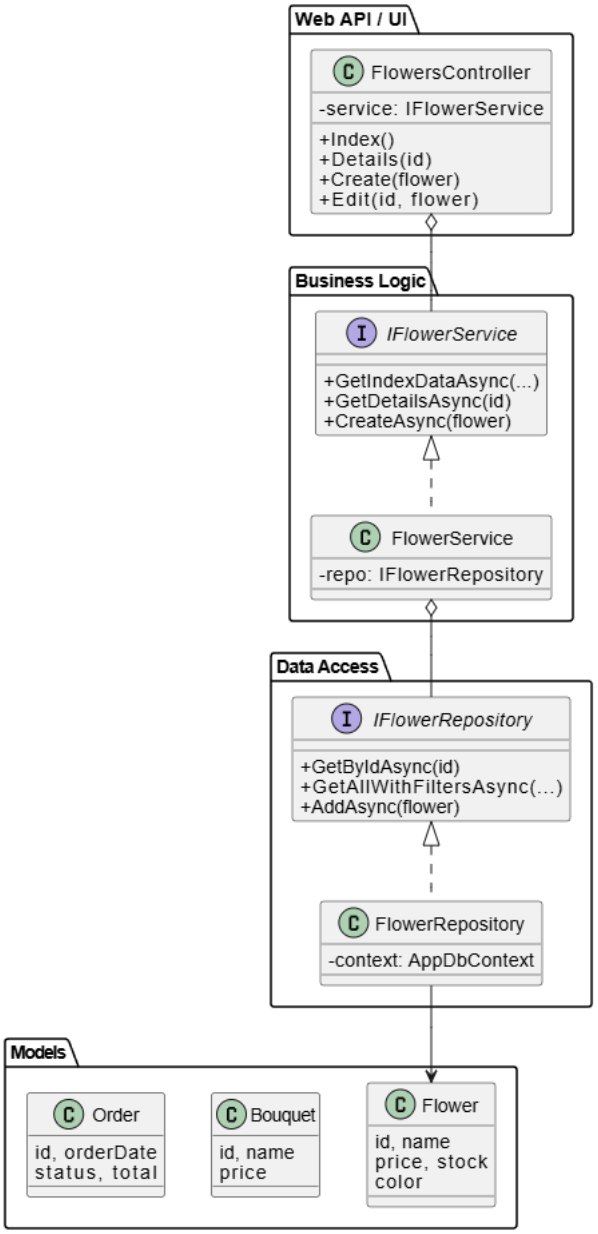


Figure 4: Class Diagram

2.7 Sequence Diagrams

Sequence diagrams illustrate how objects interact during the execution of a particular use case. They show the order of method calls and message exchanges between system components.

The order placement sequence shows the complete workflow from client request to database transaction. The CartController receives the POST request, calls OrderService to process the order, which begins a database transaction, creates the order and order items, validates stock, commits the transaction, and returns success to the controller, which then redirects the client to the order history page.

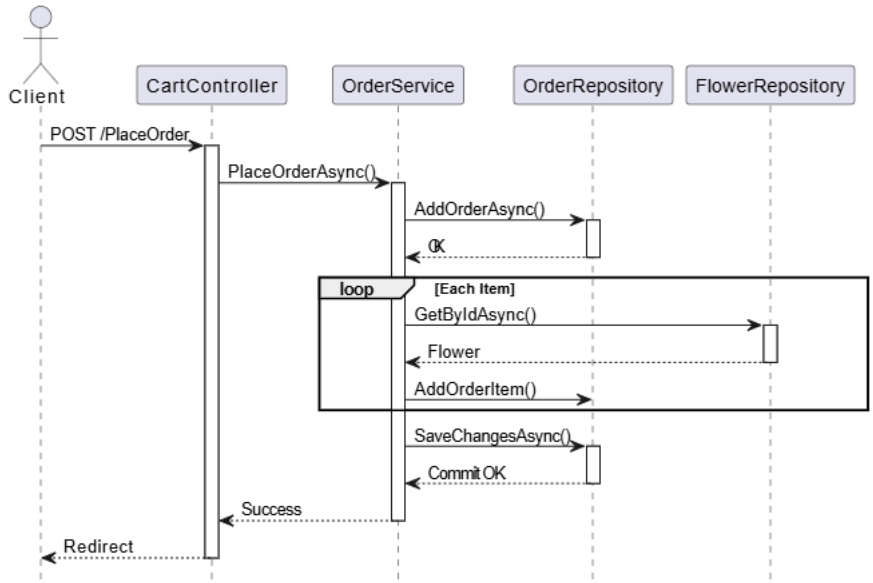


Figure 5: Class Diagram

3 Implementation Phase - Application

3.1 Tools and Technologies

The application was implemented using modern, industry-standard technologies chosen for their robustness, performance, and ecosystem support.

- **Programming Language**: C# - Chosen for strong typing, excellent performance, and seamless integration with the .NET ecosystem
- **Database Management System**: MySQL 8.0 - Selected for open-source nature, reliability, and excellent Entity Framework Core support
- **Web Framework**: ASP.NET Core 8.0 - Provides high-performance HTTP pipeline, built-in dependency injection, and comprehensive MVC pattern implementation
- **ORM**: Entity Framework Core 8.0 - Enables type-safe database operations, automatic migrations, and LINQ query capabilities
- **Frontend**: Bootstrap 5 with Razor views - Ensures responsive design and professional appearance across devices
- **Authentication**: Cookie-based authentication with session management - Simple yet secure for web applications

The combination provides excellent development productivity and runtime performance.

3.2 Application Structure

The project follows the standard ASP.NET Core folder structure with clear separation of concerns:

- **/Models**: Contains domain entities (Flower, Bouquet, Order, User, etc.) with validation attributes and navigation properties

- **/Data**: Houses AppDbContext class with Entity Framework configuration and database mappings
- **/Repositories**: Data access layer with interfaces and implementations for each entity type
- **/Services**: Business logic layer containing validation, workflow orchestration, and domain rules
- **/Controllers**: MVC controllers handling HTTP requests, authentication, and response formatting
- **/Views**: Razor templates with Bootstrap styling for user interface presentation
- **/Middleware**: Custom middleware for session-based authentication
- **/Extensions**: Helper classes for session management and utility functions

3.3 User Manual

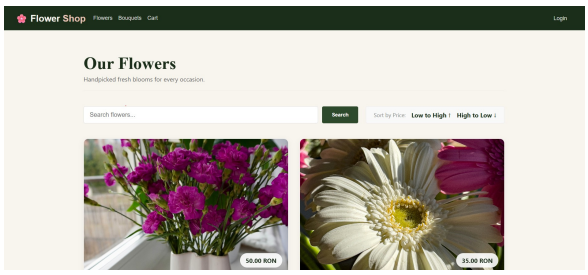
The application provides role-based interfaces tailored to different user types. The main navigation adapts based on user authentication status and role permissions.

Visitor Interface: Unauthenticated users can browse the flower catalog, view product details, and access login/registration pages. The interface emphasizes product discovery with search and filtering capabilities.

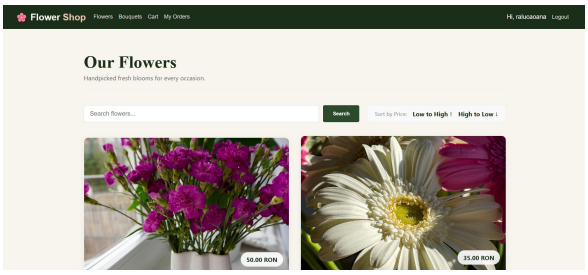
Client Interface: Authenticated customers gain access to shopping cart functionality, order placement, and personal order history. The cart appears in the navigation bar with item count and total price updates.

Florist Interface: Florists see additional menu options for product management and order processing. They can view assigned orders and update order statuses through dedicated management panels.

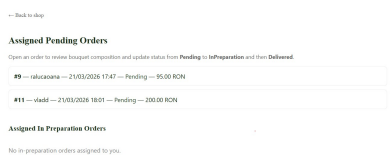
Administrator Interface: Administrators have complete system access with user management, full order oversight, and system configuration options. The admin panel provides comprehensive control over all entities and user accounts.



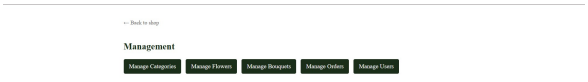
(a) Visitor Interface



(b) Client Interface



(a) Florist Interface



(b) Admin Interface

4 Testing and Validation

The project implements a comprehensive test suite using **xUnit** as the standard testing framework for ASP.NET Core applications, combined with **Moq** for creating mock objects of repository interfaces. The test project targets .NET 8 and includes **Microsoft.EntityFrameworkCore 8.0.25** specifically for mocking `IDbContextTransaction` in `OrderService` tests, ensuring version alignment with the main application.

The test project structure follows best practices with proper separation from the main application. The **Software_Design1.Tests** project is excluded from the main **Software_Design1.csproj** build configuration to prevent build conflicts, and is properly referenced in **Software_Design1.sln** solution file instead. This maintains clean separation between production code and test code.

The testing strategy focuses on unit testing the service layer in isolation, using Moq to mock all repository dependencies (IUserRepository, IOrderRepository, IFlowerRepository, IBouquetRepository). This approach ensures that business logic is tested independently of data access concerns, providing fast, reliable tests that validate all critical functionality including authentication workflows, shopping cart operations, order processing with transaction management, and user management with security constraints.